



System Architecture & Communication

ICTE 4341

Educational Cloud Networking and Management

MD. REZAUL ISLAM

Lecturer

Dept. of EdTE, BDU



Content

- ▣ System Architecture
- ▣ Architectural Patterns
- ▣ Communication in Non-distributed & Distributed System
- ▣ Communication Modes
- ▣ Communication Operation
- ▣ Communication Abstractions



System Architecture

- A **distributed system** is composed of a number of elements which are software components, processing nodes and networks.
- The software of distributed systems can become fairly complex especially in large distributed systems and its components can spread over many machines.
- System has organized in two parts- **logical organization** of software components in such a system and **physical organization**.
- Typical software architectures include the layered, object-oriented, data-centered, service-oriented and event-based architectures.
- **Note that** there is no single best architecture—the best architecture for a particular system depends on the application's requirements and the environment.

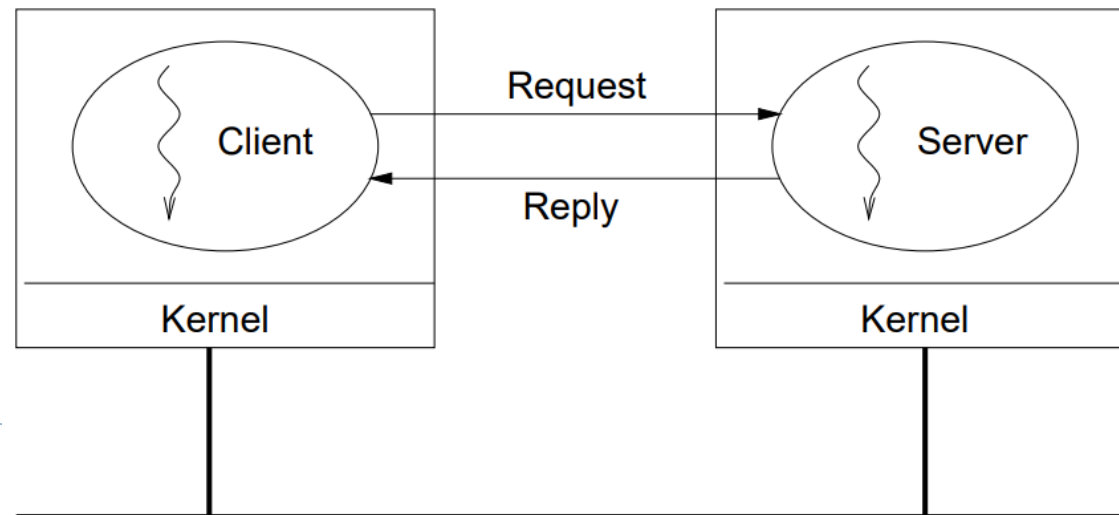


Architectural Patterns

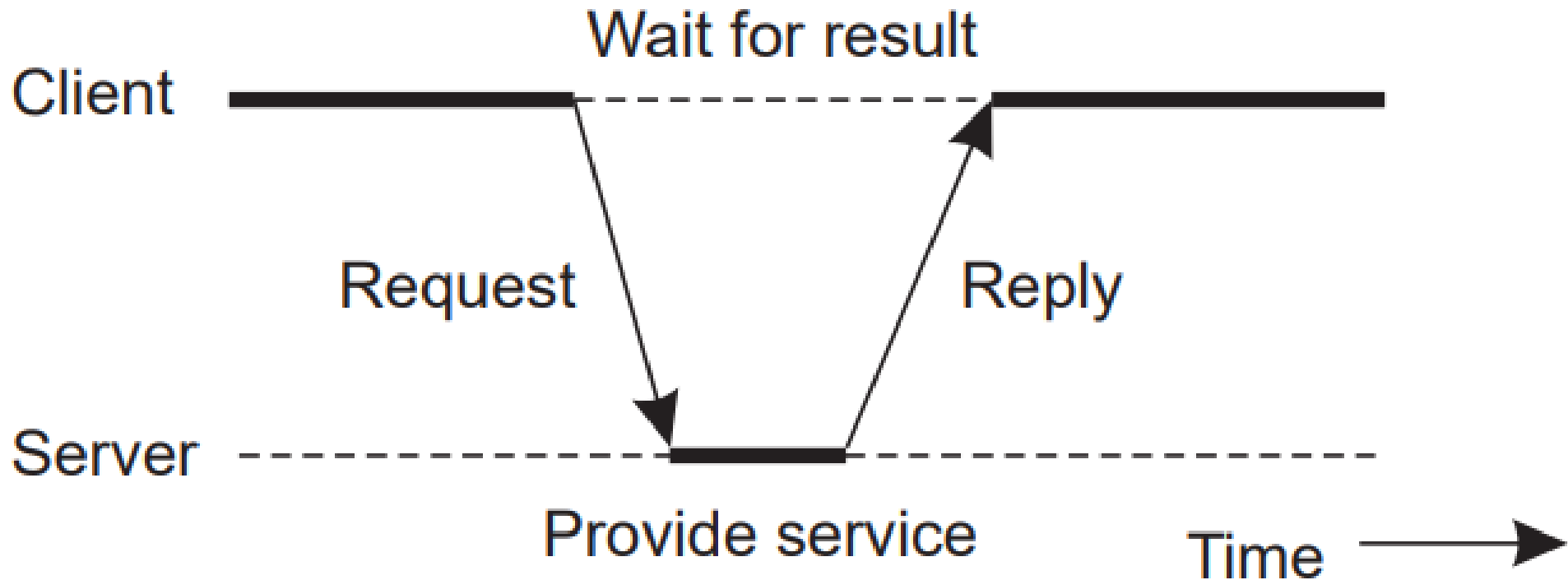


Client-Server

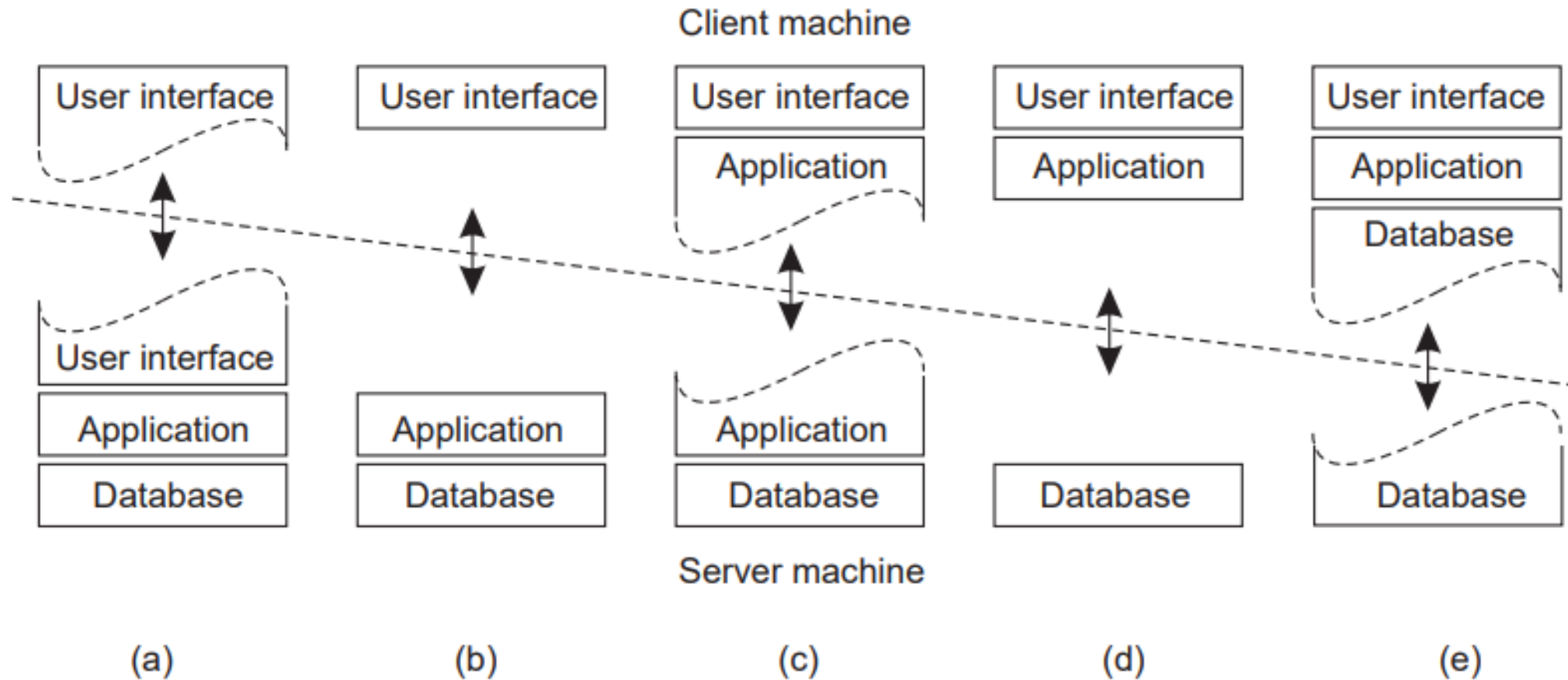
- Most common and widely used model for communication between processes.
- In this architecture one process takes on the role of a server, while all other processes take on the roles of clients.
- The server process provides a service (e.g., a time service, a database service, a banking service, etc.) and the clients are customers of that service.
- A client sends a request to a server, the request is processed at the server and a reply is returned to the client.



Client-Server from another perspective

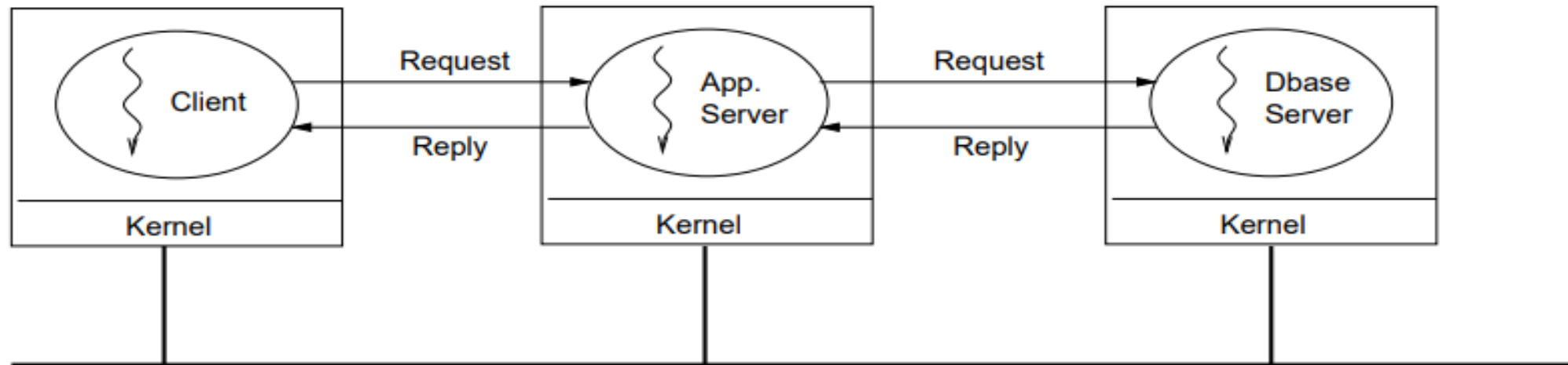


Splitting Functionality



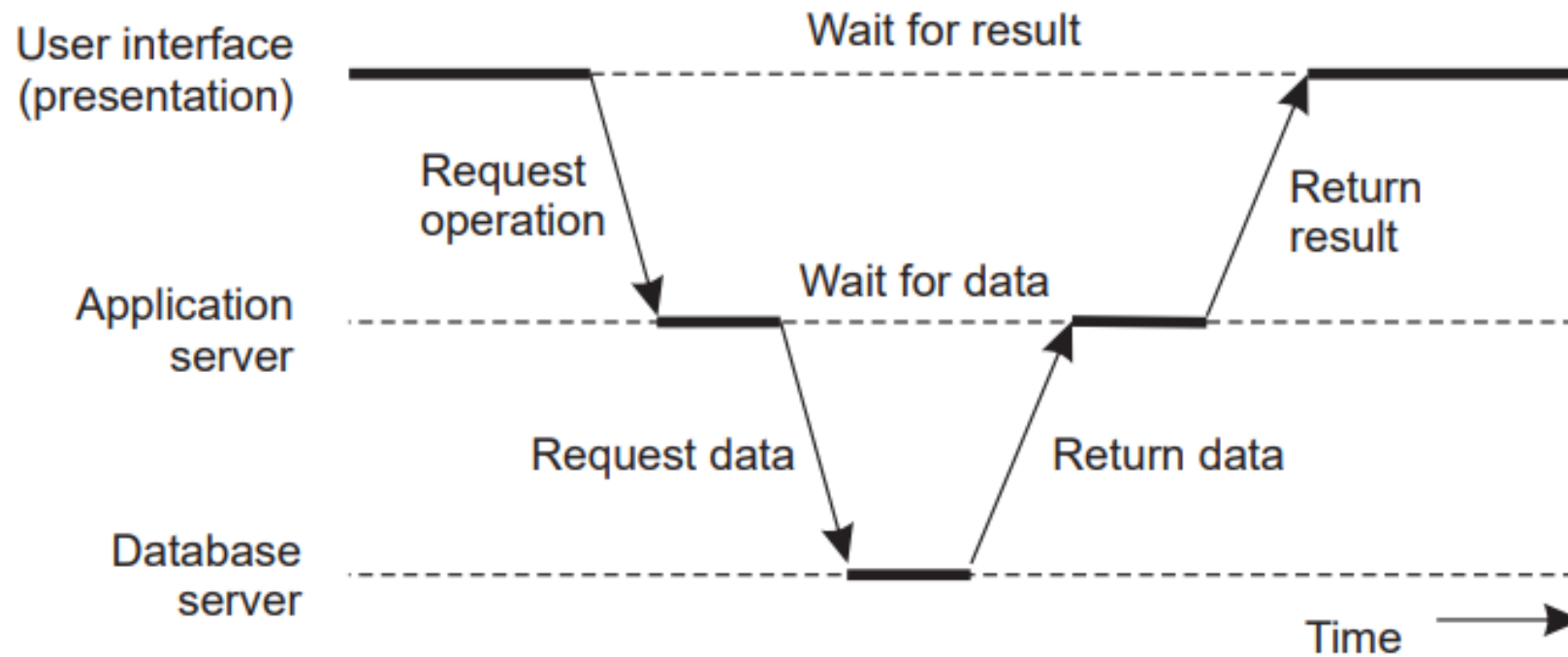
Vertical Distribution (Multi-Tier)

- **An extension of the client-server architecture**, the **vertical distribution**, or **multi-tier**, architecture distributes the traditional server functionality over multiple servers.
- A client request is sent to the first server. During processing of the request this server will request the services of the next server, who will do the same, until the final server is reached.
- In this way the various servers become clients of each other.
- Each server is responsible for a different step (or tier) in the fulfilment of the original client request.



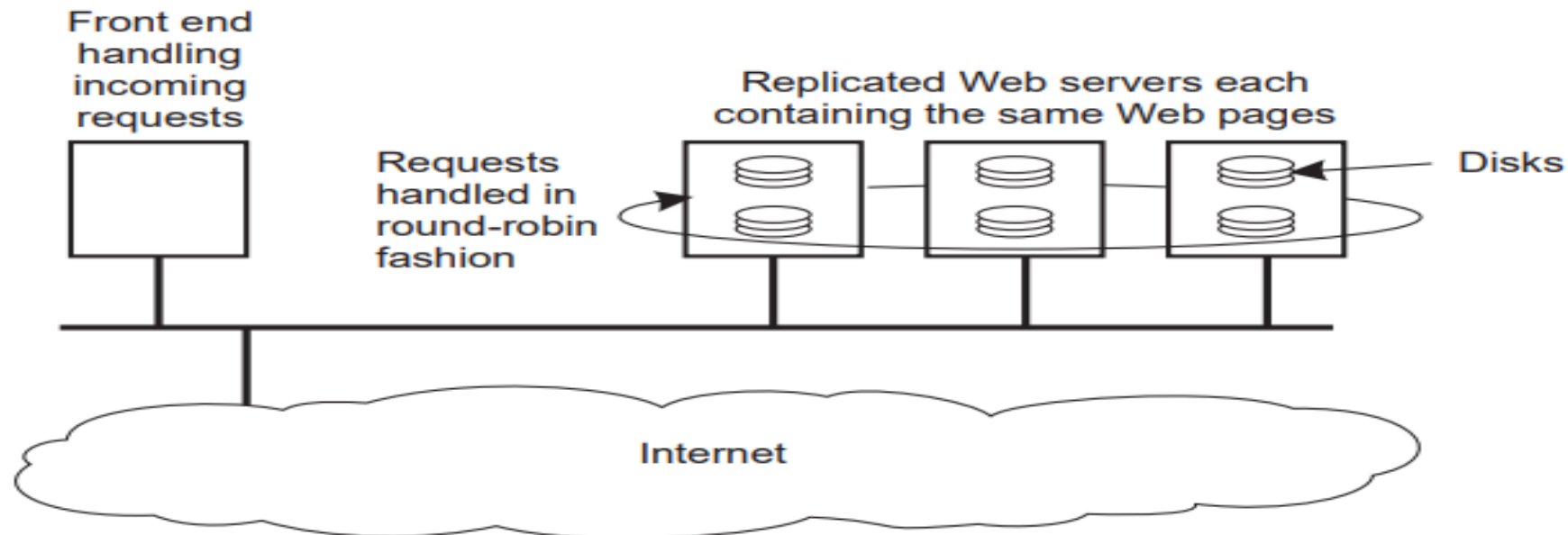
Vertical Distribution from another perspective:

- Splitting up the server functionality in this way is beneficial to a system's scalability as well as its flexibility.
- Scalability is improved because the processing load on each individual server is reduced and the whole system can therefore accommodate more users.



Horizontal Distribution

- While vertical distribution focuses on splitting up a server's functionality over multiple computers, horizontal distribution involves replicating a server's functionality over multiple computers.
- A typical example, as shown in below Figure, is a replicated Web server.
- In this case each server machine contains a complete copy of all hosted Web pages and client requests are passed on to the servers in a round robin fashion.
- The horizontal distribution architecture is generally used to improve scalability (by reducing the load on individual servers) and reliability (by providing redundancy).



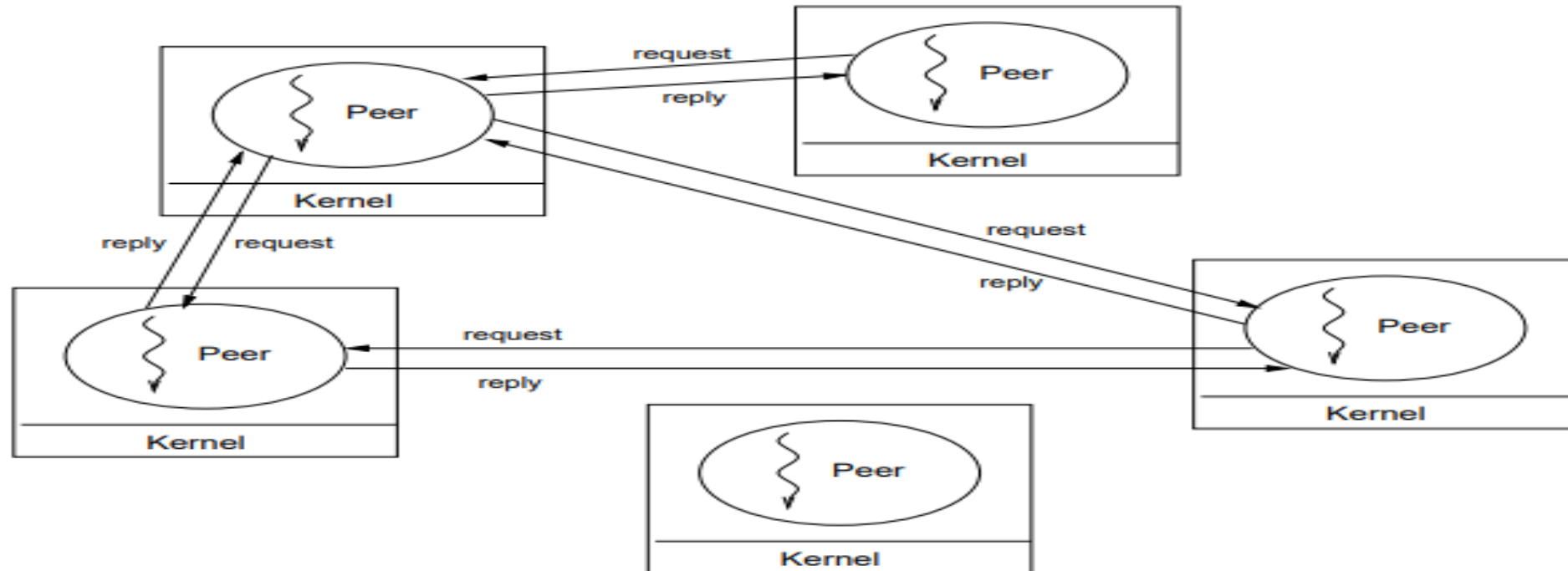
Note: Scaling Up vs Scaling Out?

- Horizontal and Vertical Distribution not the same as **Horizontal and Vertical Scaling**.
- **Vertical Scaling:** Scaling UP Increasing the resources of a single machine
- **Horizontal Scaling:** Scaling OUT Adding more machines.
- Horizontal and Vertical Distribution are both examples of this.
- Note that it is also possible to combine the **vertical and horizontal distribution models**.
- **For example,** each of the servers in the vertical decomposition can be horizontally distributed.
- **Another approach** is for each of the replicas in the horizontal distribution model to themselves be vertically distributed.



PEER TO PEER

- **Peer to Peer (P2P) architecture** takes the opposite approach and assumes that all processes play the same role, and are therefore peers of each other.
- In below figure each process acts as both a client and a server, both sending out requests and processing incoming requests.
- Well known examples of the P2P model are **file-sharing applications**.



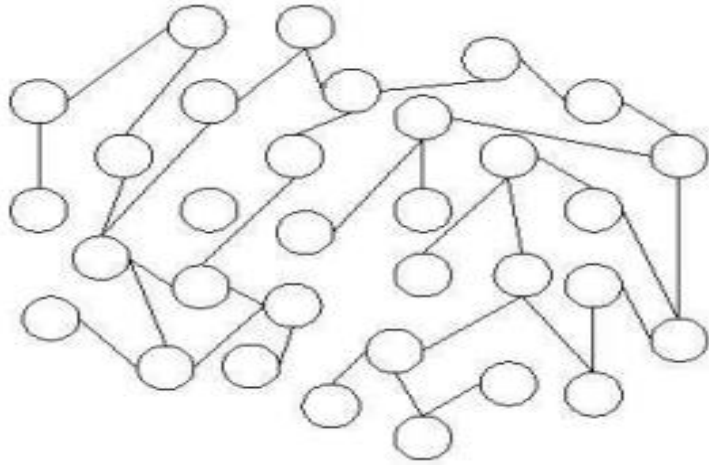
Overlay Network

How do peers keep track of all other peers?

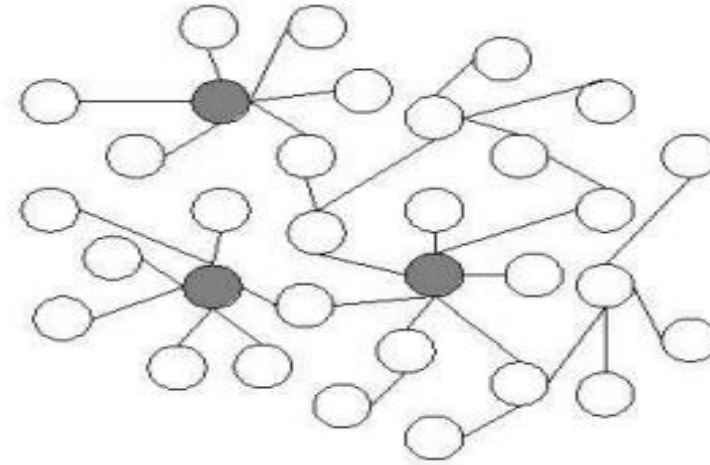
- Since huge number of participating nodes in a peer to peer network,
 - ➔ It becomes practically impossible for a node to keep track of all other nodes in the system and the information they offer.
- To reduce the problem, the nodes form an overlay network, in which nodes form a virtual network among themselves and only have direct knowledge of a few other nodes.
- When a node wishes to send a message to an arbitrary other node it must first locate that node by propagating a request along the links in the overlay network. Once the destination node is found, the two nodes can typically communicate directly.
- **There are two key types of overlay networks:** Structured and Unstructured



Unstructured Overlay Network



(a) Random network

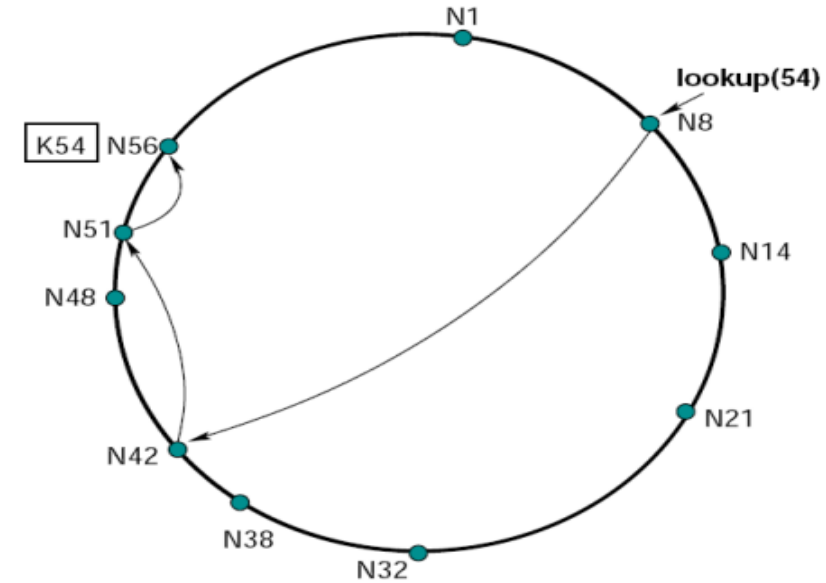
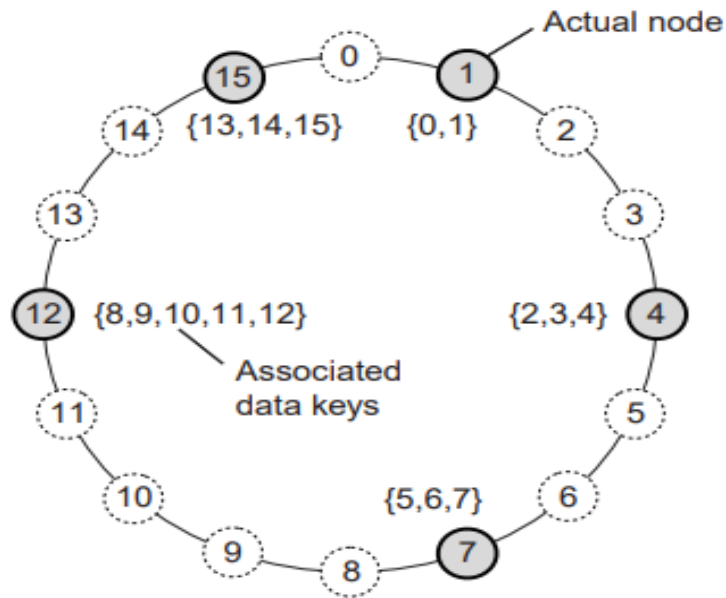


(b) Scale-free network

- Data stored at random nodes
- **Partial view:** random node's list of neighbours.
- Exchange partial views with neighbours to update and creating a new neighbour list for themselves.



Structured Overlay Network



- Choice of a node's neighbours is determined according to a specific structure.
- In a **distributed hash table** nodes work together to implement a hash table.
- Each node is responsible for storing the data associated with a range of identifiers.
 - ⇒ Node is responsible for data that falls in its range
 - ⇒ Search is routed to appropriate node

Hybrid Architectures

- Many more architectures can be designed by combining the previously described architectures in different ways and result in what are called **hybrid architectures**. A few examples are:

- ⇒ **Superpeer networks**

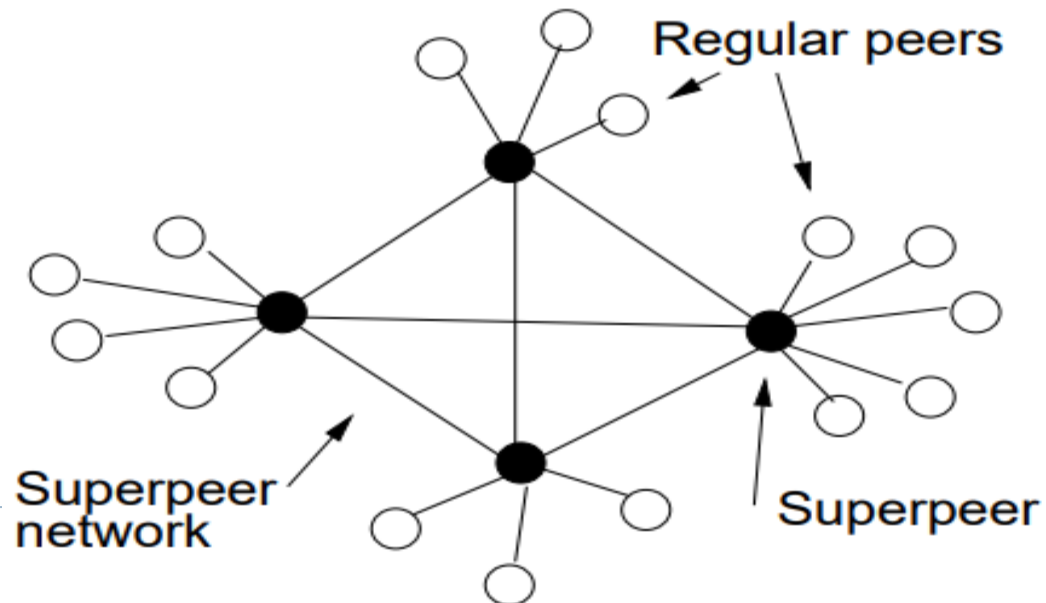
- ⇒ **Collaborative distributed systems**

- ⇒ **Edge-server networks**



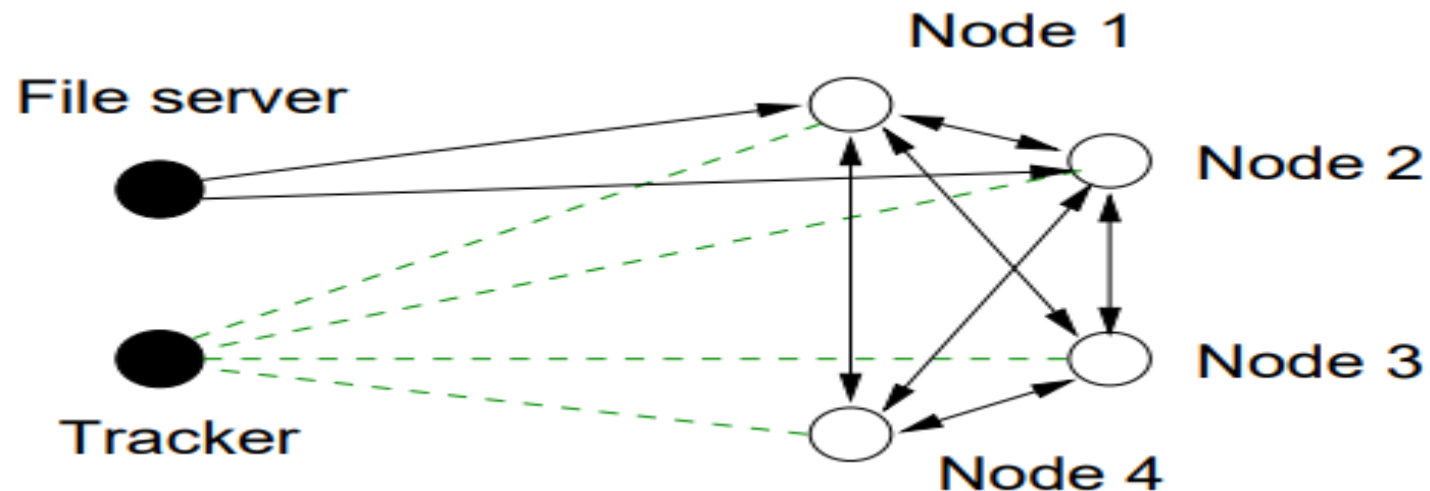
Superpeer networks

- In this architecture a few superpeers form a peer to peer network, while the regular peers are clients to a **superpeer**.
 - ⇒ Regular peers are clients of superpeers
 - ⇒ Superpeers are servers for regular peers
 - ⇒ Superpeers are peers among themselves
 - ⇒ Superpeers may maintain large index, or act as brokers
 - ⇒ **Example:** Skype



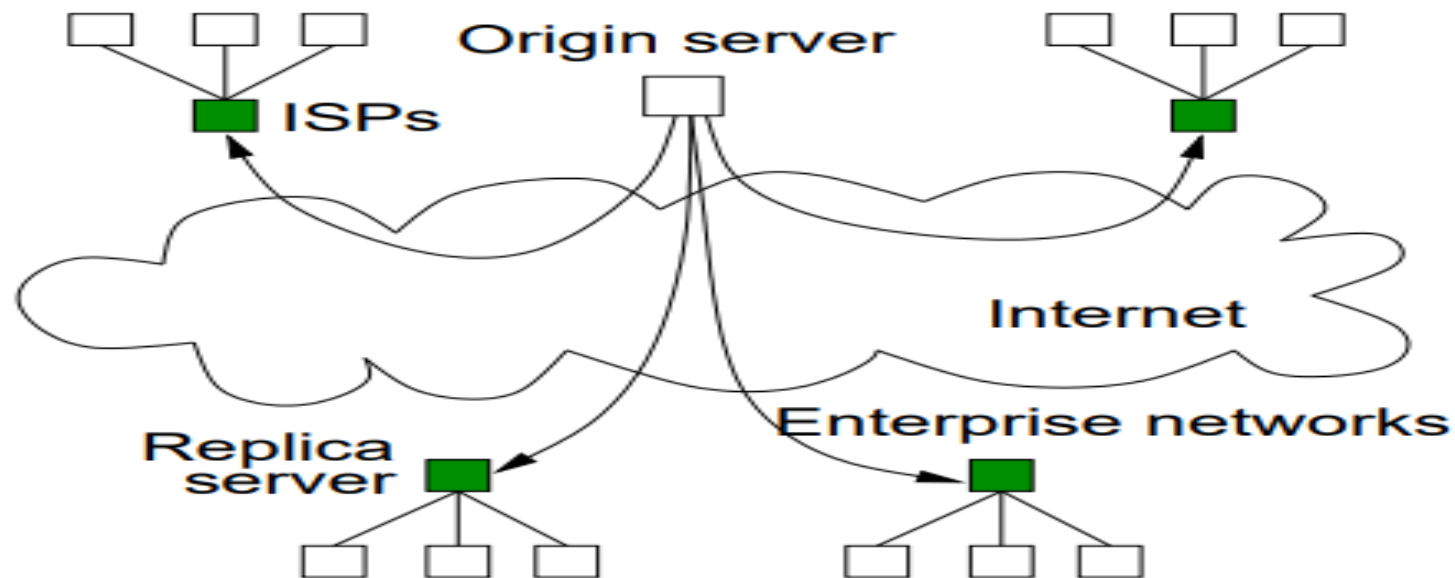
Collaborative distributed systems

- In this systems, peers support each other to deliver content in a peer to peer like architecture, while they use a client server architecture for the initial setup of the network. **Ex:** BitTorrent
 - ⇒ Node downloads chunks of file from many other nodes
 - ⇒ Node provides downloaded chunks to other nodes
 - ⇒ Tracker keeps track of active nodes that have chunks of fil



Edge-server networks

- Client nodes (e.g., home users or an enterprise's employees) then access the nearby edge servers instead of the original server (which may be located far away).
 - ⇒ Servers placed at the edge of the network
 - ⇒ Servers replicate content
 - ⇒ Mostly used for content and application distribution
 - ⇒ **Content Distribution Networks:** Akamai, CloudFront, CoralCDN



Communication

- **Why Communication need?**
- Cooperating processes need to communicate.
 - ⇒ For synchronization and control
 - ⇒ To share data
- A key property of all distributed systems is that they consist of separate processes that communicate in order to get work done.
- How do communication between processes on a single computer (i.e., a uniprocessor or multiprocessor)?



Communication in Non-distributed System



Non-Distributed System

- **Two approaches to communication:**

- **Shared memory**

- ⇒ **Direct memory access (Threads):** Multiple threads of control share a single address space. They can freely access and modify each other's memory.

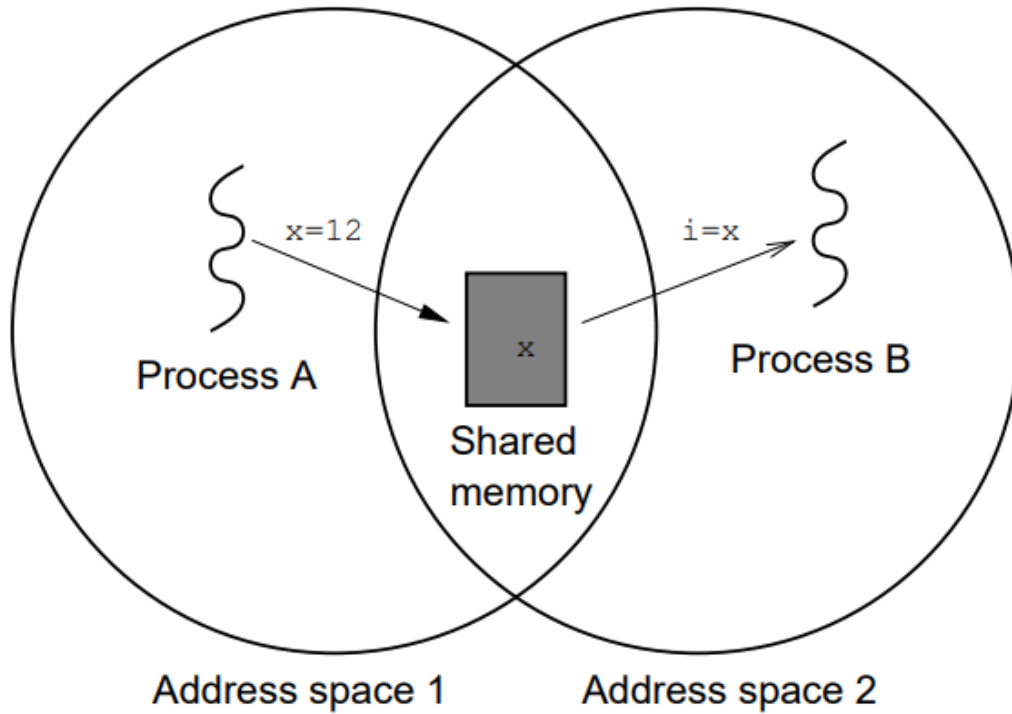
- ⇒ **Mapped memory (Processes):** In the process model, each process is associated with a single private address space. They are prevented from accessing each other's memory.

- ⇒ Some system provide both where each process can contain multiple threads, which means that threads can only freely access the memory of other threads in the same process.

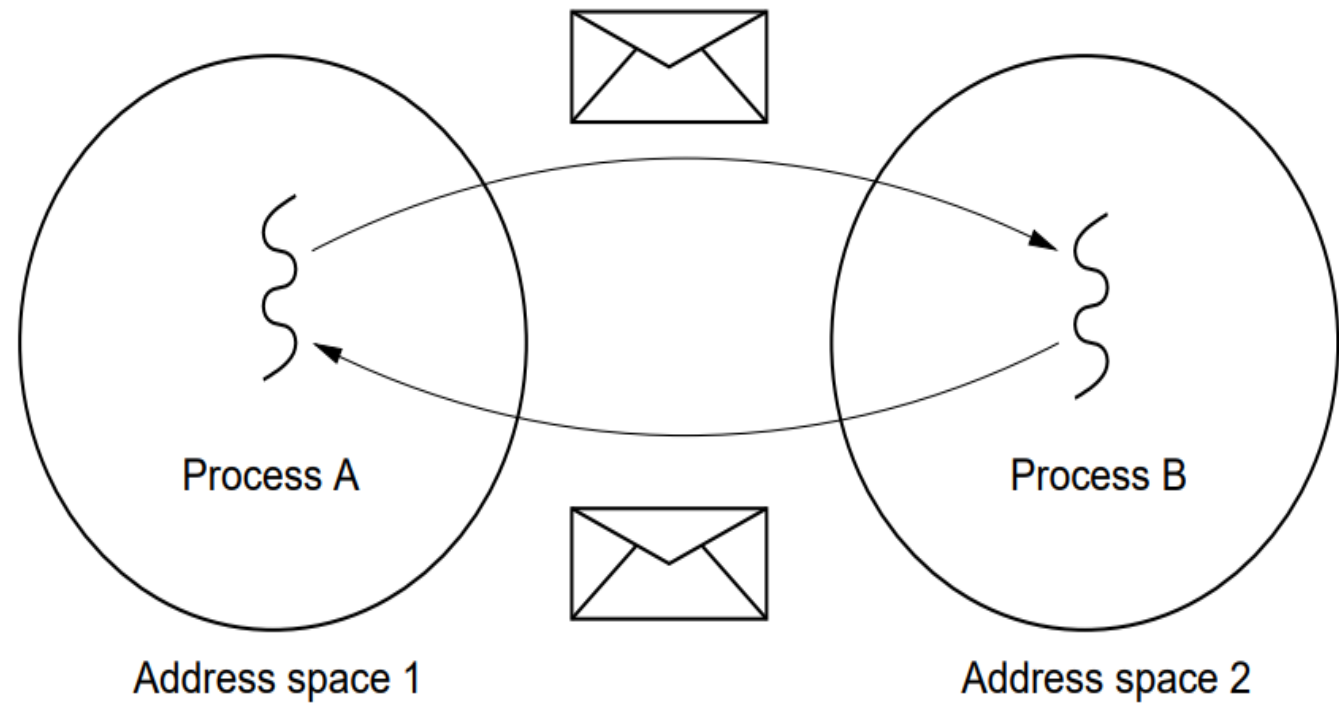
- **Message passing**



Non-Distributed System



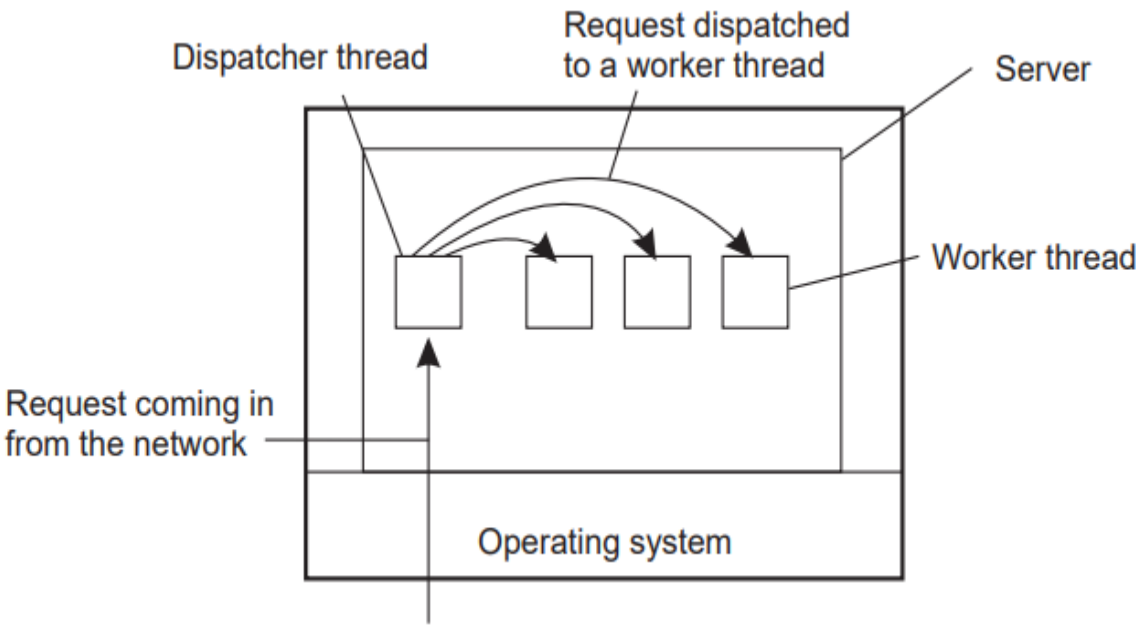
Shared Memory



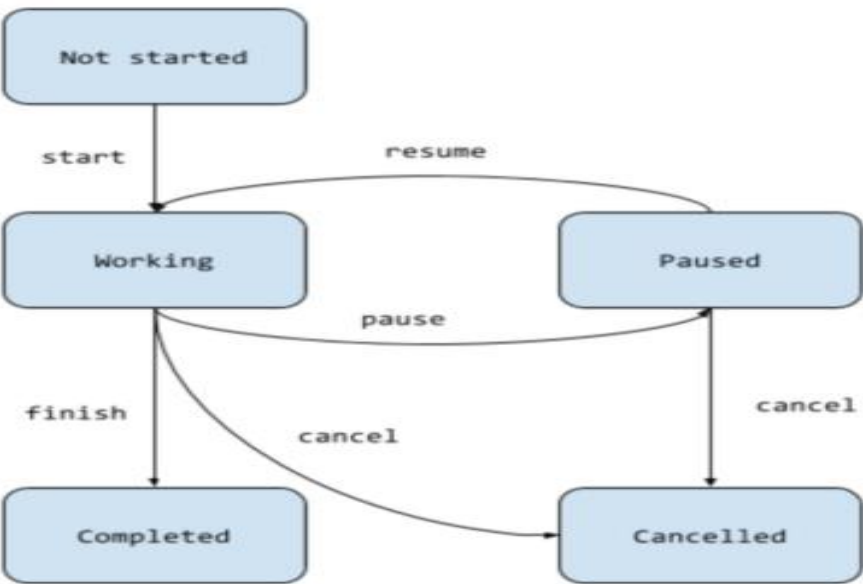
Message Passing



Server Design



Multiple Threads



Finite-state machine

Model	Characteristics
Single-threaded process	No parallelism, blocking system calls
Multiple Threads	Parallelism, blocking system calls
Finite-state machine	Parallelism, non-blocking system calls

Stateful Vs Stateless Servers

★ **Stateful Server:**

- ⇒ Keeps persistent information about clients
- ✓ Improved performance
- ✓ Expensive crash recovery
- ✓ Must track clients

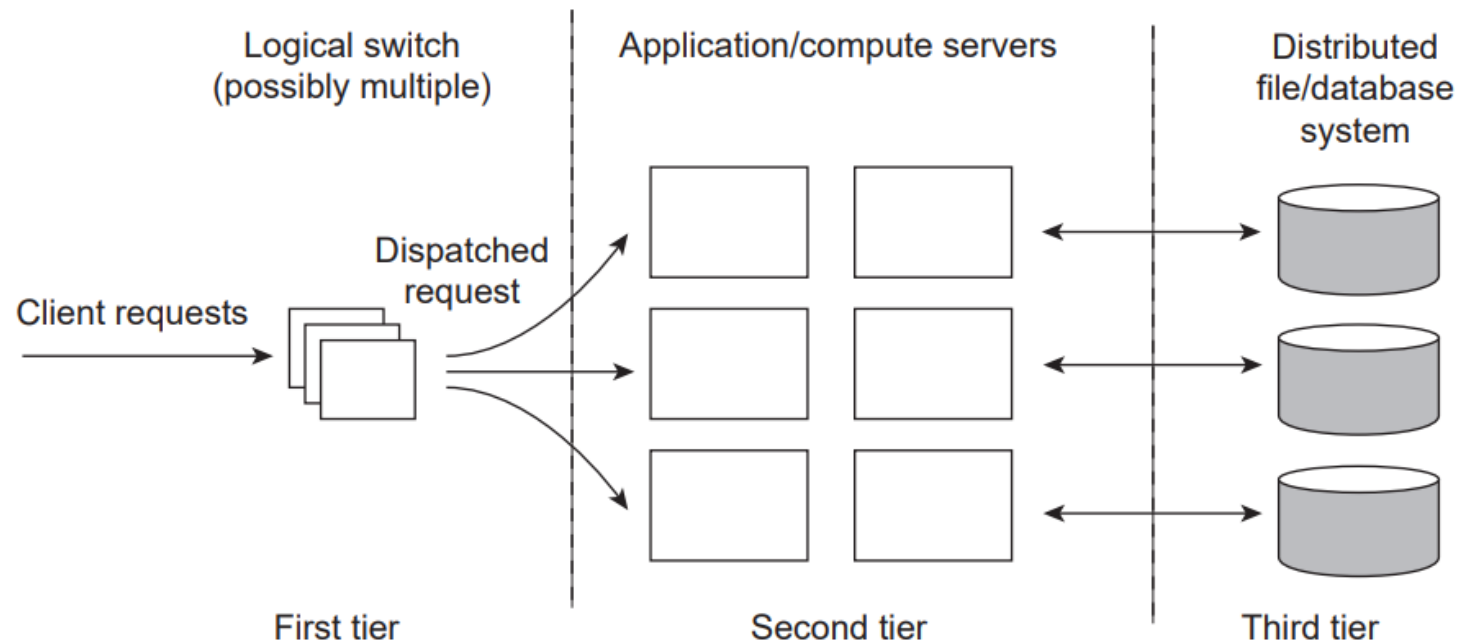
★ **Stateless Server:**

- ⇒ Does not keep state of clients
- ⇒ Can change own state without informing clients
- ✓ Don't need to worry about restoring client state after a crash.
- ✓ Easy to replicate.
- ✓ Requires more communication since clients have to resend their state information with every request.



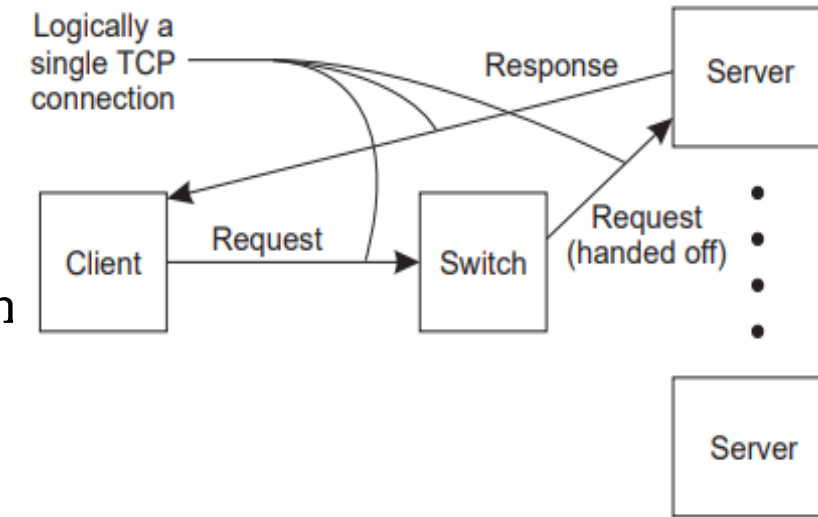
Clustered Servers

- A server in client-server is not a single machine but a collection of machines that act as a **clustered server**.
- Machines (and processes) in a cluster are assigned dedicated roles including a **logical switch**, compute (or application logic) servers, and file or database servers.



Request Switching

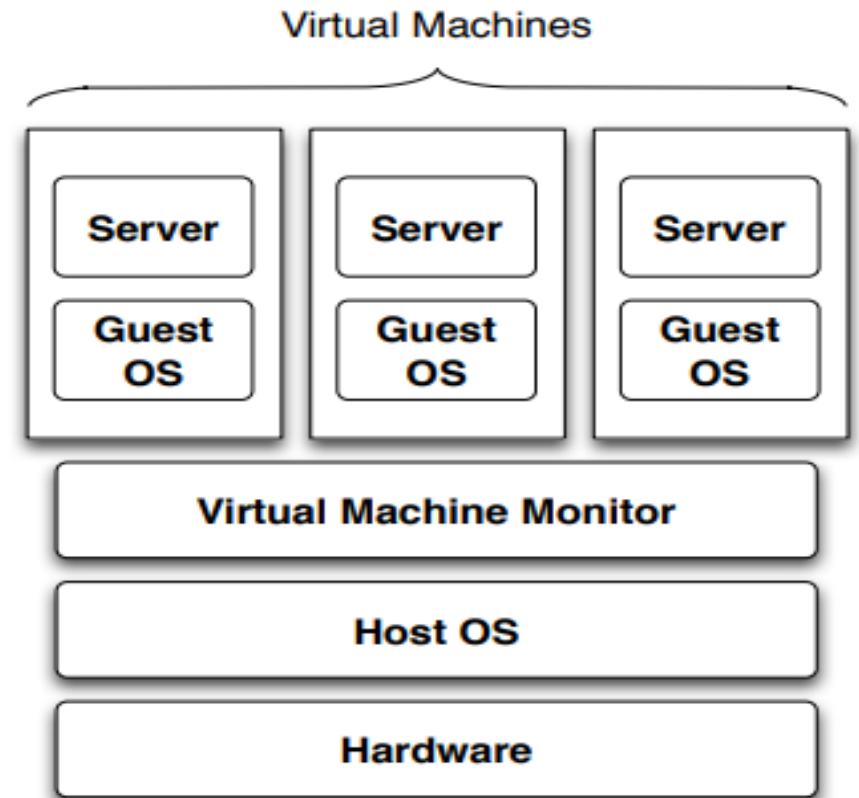
- The role of the switch is to receive client requests and route them to appropriate servers in the cluster.
- **Transport-layer switch**, reroutes TCP connections to other servers. Decision regarding which server to route the request depends on system load.
- **Application layer switch**: Analyze requests and forward to appropriate server
 - ⇒ For example, HTTP requests for HTML files could be routed to one set of servers, while requests for images could be served by other servers.
- **DNS server act as a switch** by returning different IP addresses for a single host name using round-robin algo.



Transport-layer Switch

Virtualisation

- In modern systems separate virtual machines are hosted on a **single physical machine**.
- This allows consolidation of many servers on a single machine, while providing isolation between them.
- Since virtual machines can be stopped, migrated, and restarted, virtualisation provides a good basis for **code mobility and load balancing**.



Code Mobility

- Why move code?
 - ⇒ Optimise computation (load balancing)
 - ⇒ Optimise communication
- **Two types of code mobility:** weak mobility and strong mobility.
 - ⇒ **Weak mobility** transfer only code
 - ⇒ **Strong mobility** transfer code and execution segment



Communication in Distributed System



Communication in Distributed System

- Previous slides assumed a **uniprocessor** or a **multiprocessor**.
- In a distributed system (multicomputer) things change:
- **Shared Memory:**
 - ⇒ There is no way to physically share memory
- **Message Passing:**
 - ⇒ Over the network
 - ⇒ Introduces latencies
 - ⇒ Introduces higher chances of failure
 - ⇒ Heterogeneity introduces possible incompatibilities



Message Passing in Distributed System

- **Basics:** Message passing involves two operations
 - ⇒ send()
 - ⇒ receive()
- There are a number of important issues to consider when dealing with processes that communicate using message passing. These variations are:
 - ⇒ Connection oriented vs Connectionless
 - ⇒ Point-to-point vs Group
 - ⇒ Synchronous vs Asynchronous
 - ⇒ Buffered vs Unbuffered
 - ⇒ Reliable vs Unreliable
 - ⇒ Message ordering guarantees



Communication Modes

- **Data-Oriented vs Control-Oriented Communication:**
- **Data-oriented communication**
 - ⇒ Facilitates data exchange between threads
 - ⇒ Shared address space, shared memory & message passing
- **Control-oriented communication**
 - ⇒ Associates a transfer of control with communication
 - ⇒ Active messages, remote procedure call (RPC) & remote method invocation (RMI)



Communication Operation

- **Synchronous vs Asynchronous Communication:**
- **Synchronous**
 - ⇒ Sender blocks until message received. Often sender blocked until message is processed and a reply received.
 - ⇒ Sender and receiver must be active at the same time
 - ⇒ Receiver waits for requests, processes them (ASAP), and returns reply
 - ⇒ Client-Server generally uses synchronous communication
- **Asynchronous**
 - ⇒ Sender continues execution after sending message (does not block waiting for reply)
 - ⇒ In the **buffered case**, a message will be stored if the receiver is not able to pick it up right away.
 - ⇒ In the **unbuffered case** the message will be lost.
- **When is Synchronous suitable? Asynchronous?**



Communication Operation

- **Transient vs Persistent Communication:**

- **Transient**

- ⇒ Message will only be delivered if a receiver is active.
- ⇒ If there is no active receiver process (i.e., no one interested in or able to receive messages) then an undeliverable message will simply be dropped.
- ⇒ **Example:** HTTP request

- **Persistent**

- ⇒ Message stored (somewhere) until receiver can accept it
- ⇒ **Example:** email



Communication Operation

- **Provider-Initiated vs Consumer-Initiated Communication**

- **Provider-Initiated**

- ⇒ Message sent when data is available

- ⇒ Example: notifications

- **Consumer-Initiated**

- ⇒ Request sent for data

- ⇒ **Example:** HTTP request



Communication Operation

- **Direct-Addressing vs Indirect-Addressing Communication:**

- **Direct-Addressing**

- ⇒ Message sent directly to receiver

- ⇒ **Example:** HTTP request

- **Indirect-Addressing**

- ⇒ Message not sent to a particular receiver

- ⇒ **Example:** broadcast, publish/subscribe



Communication Operation

- **Reliable vs Unreliable Communication:**

- **Reliable Communication**

⇒ Processes can assume that a message that is sent will actually arrive at the destination (as long as the destination process is there to receive it).

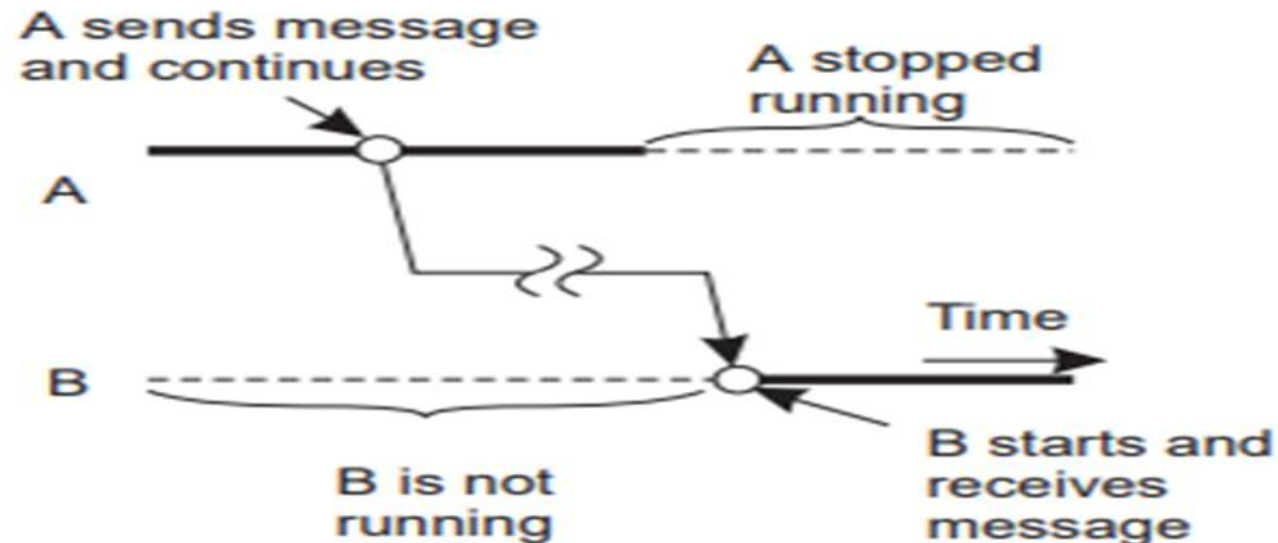
- **Unreliable Communication**

⇒ messages may get lost and processes have to deal with it.



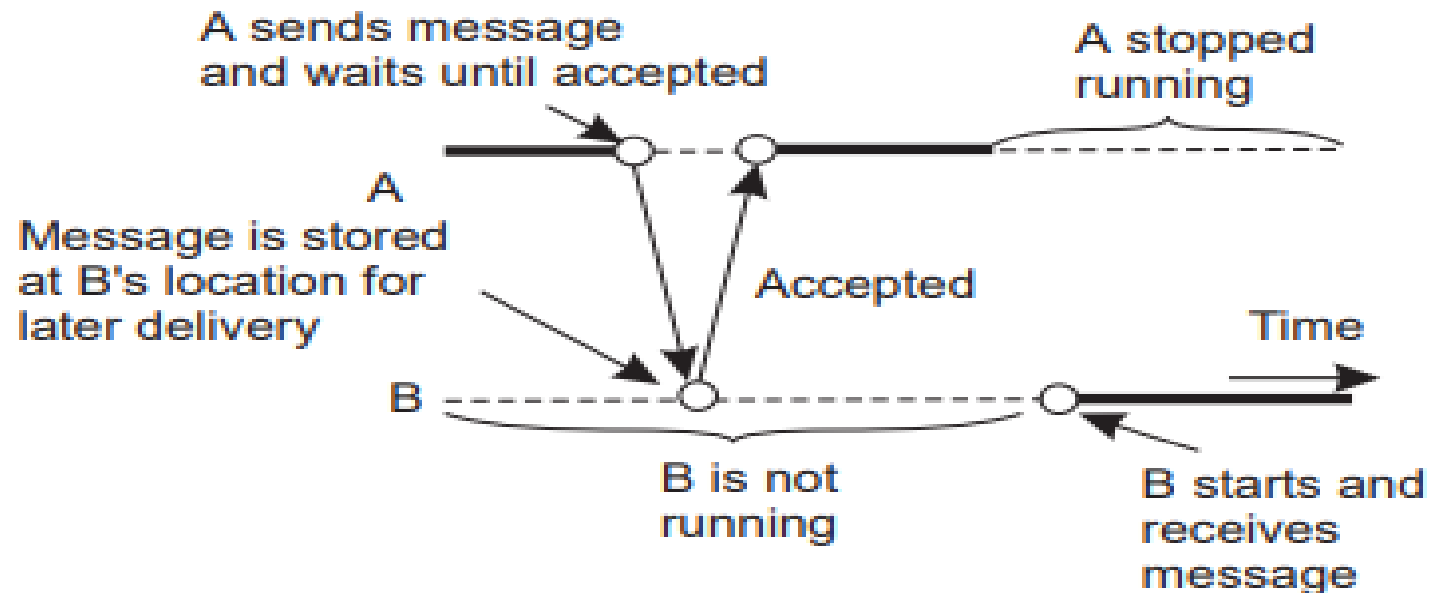
Combinations - Persistence and Synchronicity

1. **Persistent Asynchronous Communication:** A sends a message and keeps executing without blocking. The message sent from A will take an arbitrary amount of time to reach B. A may or may not be running by the time the message reaches B. Disks or multiple memory queues could be used for storage at B's side.
 - There is a guarantee that the message will eventually reach B. B receives this request and processes it then.
 - **Example:** Emails can take an arbitrary amount of time to reach and when the receiver reads the email, the sender might not even be running.



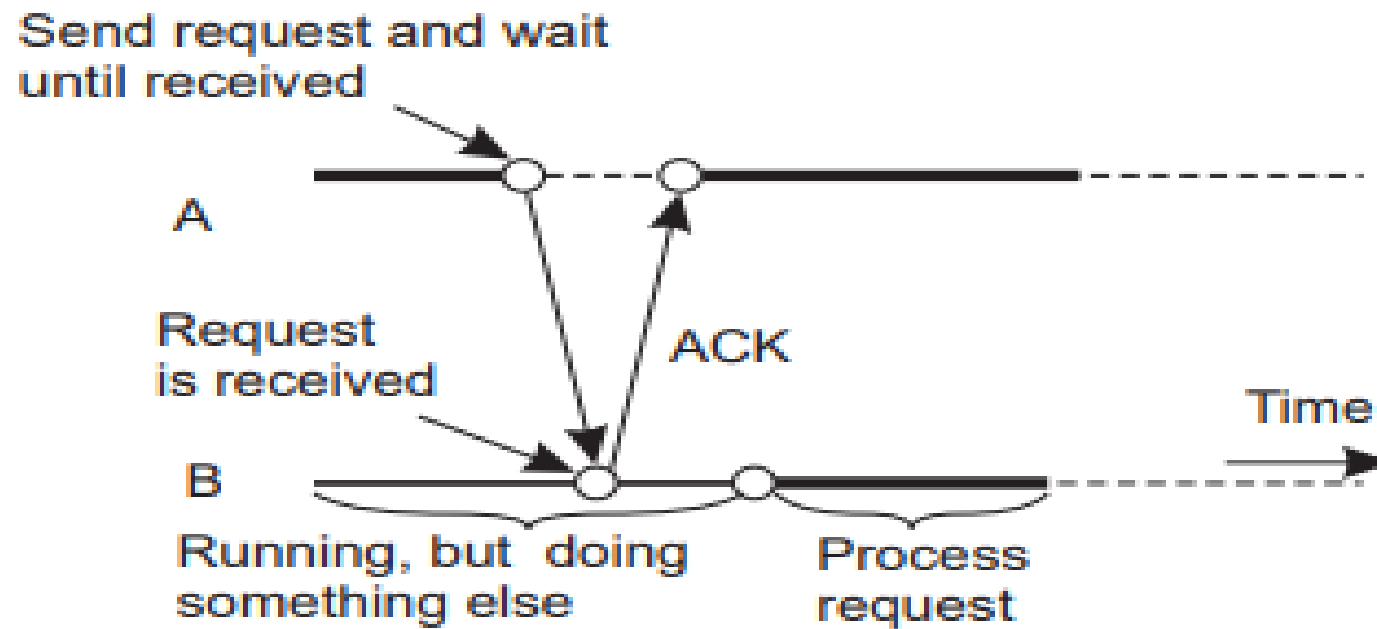
Combinations - Persistence and Synchronicity

2. **Persistent Synchronous Communication:** Here, A sends a message, and is then blocked (depicted by the dotted line). The acknowledgement is for receipt (not delivery/response). Because this model is persistent, the message may stay in B's queue (or in any router along the way) for an arbitrary amount of time.
- **Example:** Messaging and chat applications; Many messaging systems are persistent, and can tell us the delivery status.



Combinations - Persistence and Synchronicity

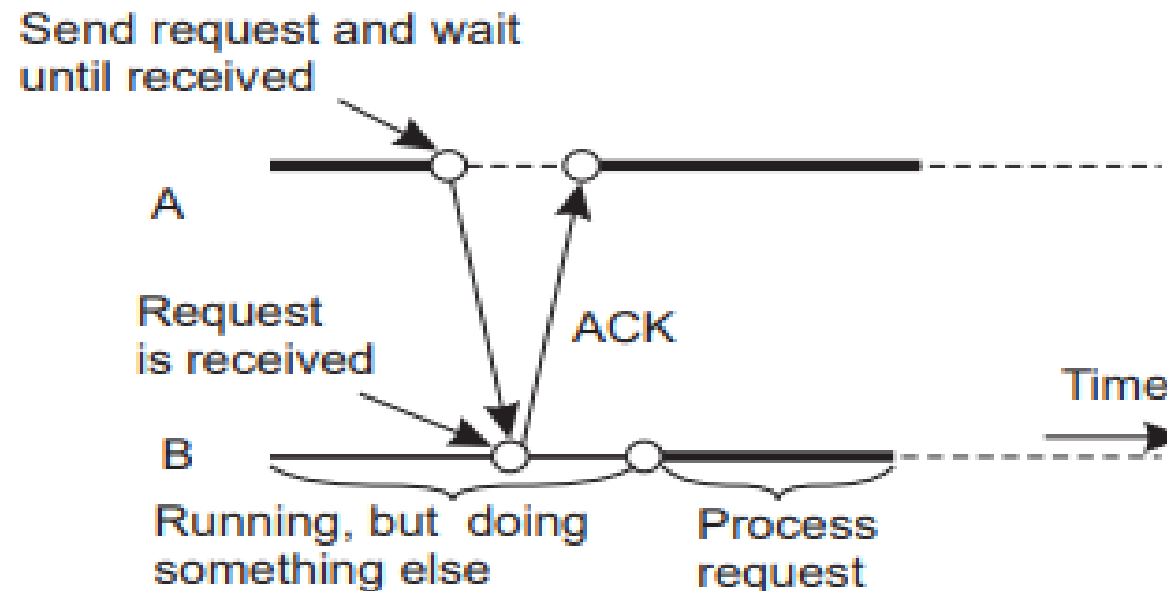
3. **Transient Asynchronous Communication:** A sends the message and continues execution (nonblocking). B has to be running, because if it is not running the message will be discarded. Even if any router along the way is down, the message will be discarded.
- **Example:** UDP



Combinations - Persistence and Synchronicity

4. Transient Synchronous Communication

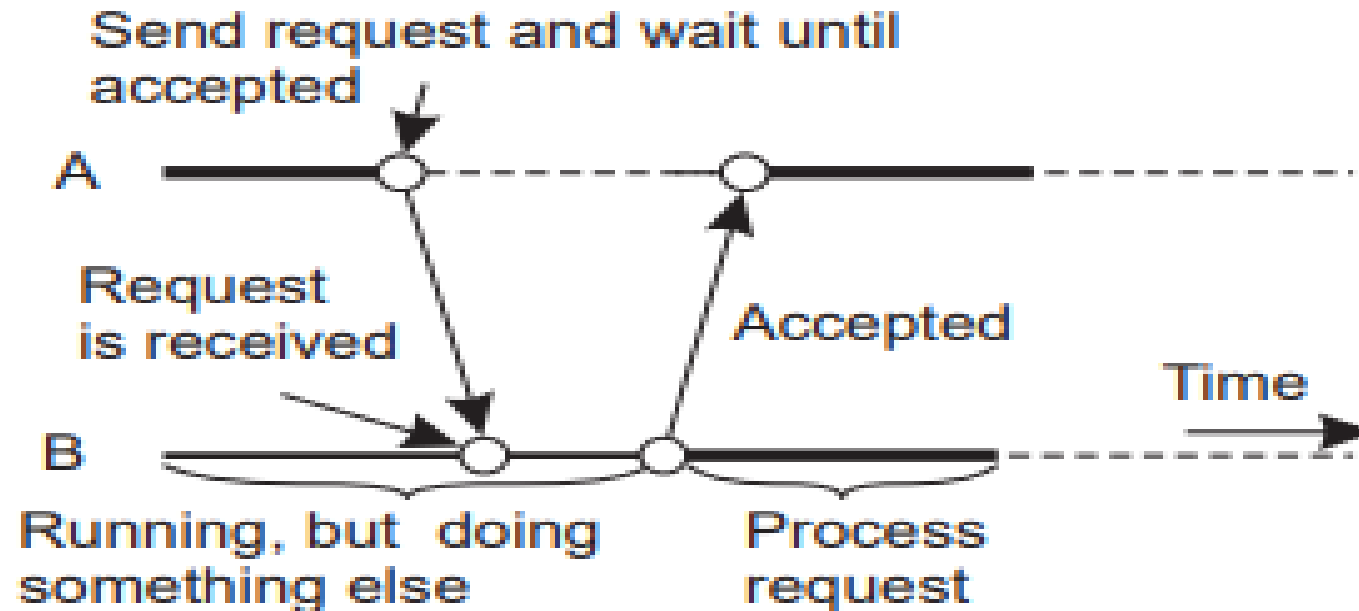
- a) **Receipt-based Transient Synchronous Communication:** The acknowledgement sent back from the receiver indicates that the message has been received by the other end. The receiver might be working on some other process.
- A's message to B is blocking (synchronous) until an ack is received. This ack simply tells us that the message was received at the other end. It does not tell us anything about whether the process has started.



Combinations - Persistence and Synchronicity

4. Transient Synchronous Communication

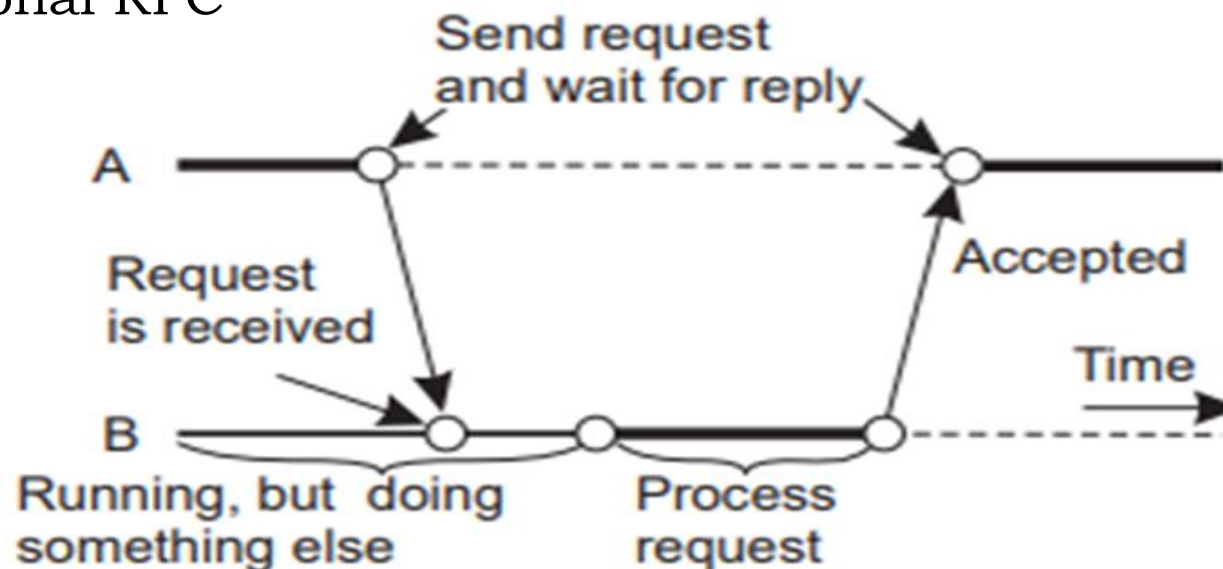
- b) Delivery-based transient synchronous communication:** The acknowledgement comes back to the sender when the other end actually takes control of the message.
- A will resume running when B takes the delivery of the message. The ack comes a little bit later than the previous method.
 - **Example:** Asynchronous RPC



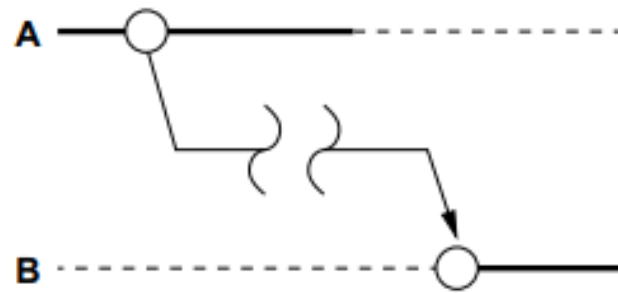
Combinations - Persistence and Synchronicity

4. Transient Synchronous Communication

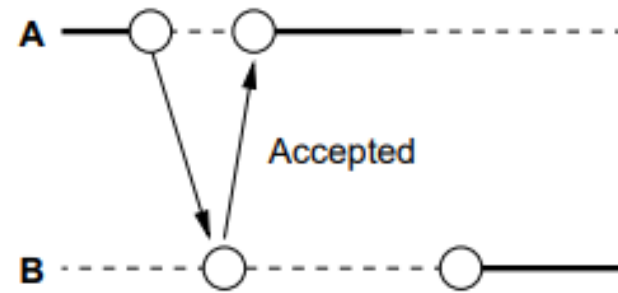
- c) **Response-based transient synchronous communication:** The sender blocks until the receiver processes the request and sends back a response.
- A resumes execution upon receiving a response. That is, not only has the message been delivered, but it has been processed and the ack comes back in the form of a reply.
 - **Example:** Traditional RPC



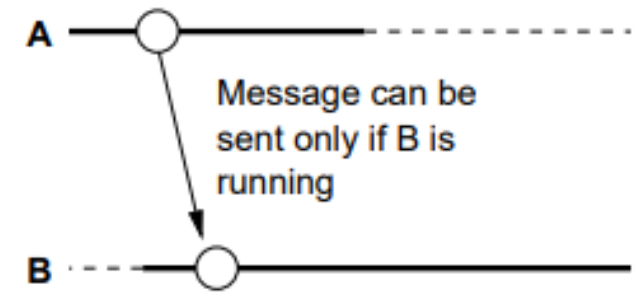
Combinations - Persistence and Synchronicity



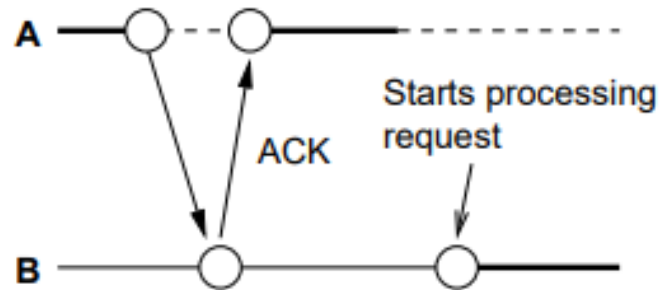
Persistent Asynchronous



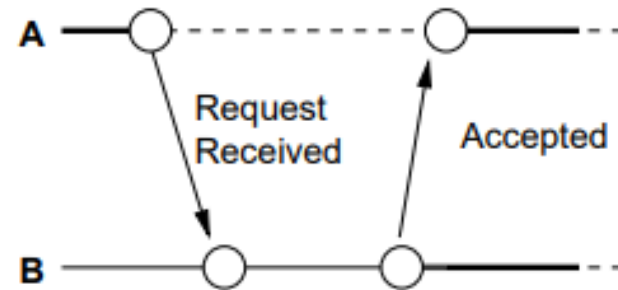
Persistent Synchronous



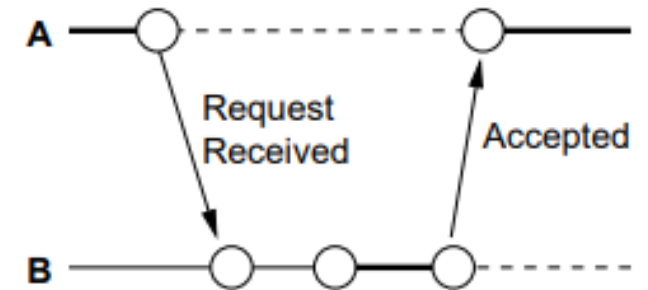
Transient Asynchronous



Transient Synchronous
(Receipt Based)



Transient Synchronous
(Delivery Based)



Transient Synchronous
(Response Based)



Communication Abstractions

- Abstractions above simple message passing make communication easier for the programmer writing distributed applications
- Like higher level programming languages make programming easier by providing abstractions above assembly language
- Abstractions used as core foundations of most **middleware systems**.
- **Provided by higher level APIs**
 - ❶ Message-Oriented Communication
 - ⇒ Provided by message-oriented middleware (MOM)
 - ⇒ **Example of MOM** : Request-Reply, Remote Procedure Call (RPC) & Remote Method Invocation (RMI)
 - ❷ Group Communication
 - ❸ Event-based Communication
 - ❹ Shared Space



1. Message-Oriented Communication

- Its goal is to make the use of flexible message passing easier.
- Traditional send()/receive() provides:
 - ⇒ Asynchronous and Synchronous communication
 - ⇒ Transient communication
- What more does it provide than send()/receive()?
 - ⇒ Persistent communication (Message queues)
 - ⇒ Hides implementation details
 - ⇒ **Marshalling**: Formatting a collection of data structures into an external data representation type appropriate for transmission in a message.
 - ⇒ Programmers to use message passing without having to be aware of what platforms their software will run
- **Example**: Message Passing Interface (MPI), Message queuing system (MQ), Remote Procedure Call (RPC)



Example: Message Passing Interface (MPI)

- Its goal is to make the use of flexible message passing easier.
- Traditional send()/receive() provides:
 - ⇒ Asynchronous and Synchronous communication
 - ⇒ Transient communication
- What more does it provide than send()/receive()?
 - ⇒ Designed for parallel applications
 - ⇒ Makes use of available underlying network
 - ⇒ No persistent communication
 - ⇒ Primitives for all forms of transient communication
 - ⇒ It makes use of available networking protocols



Example: Message queuing system (MQ)

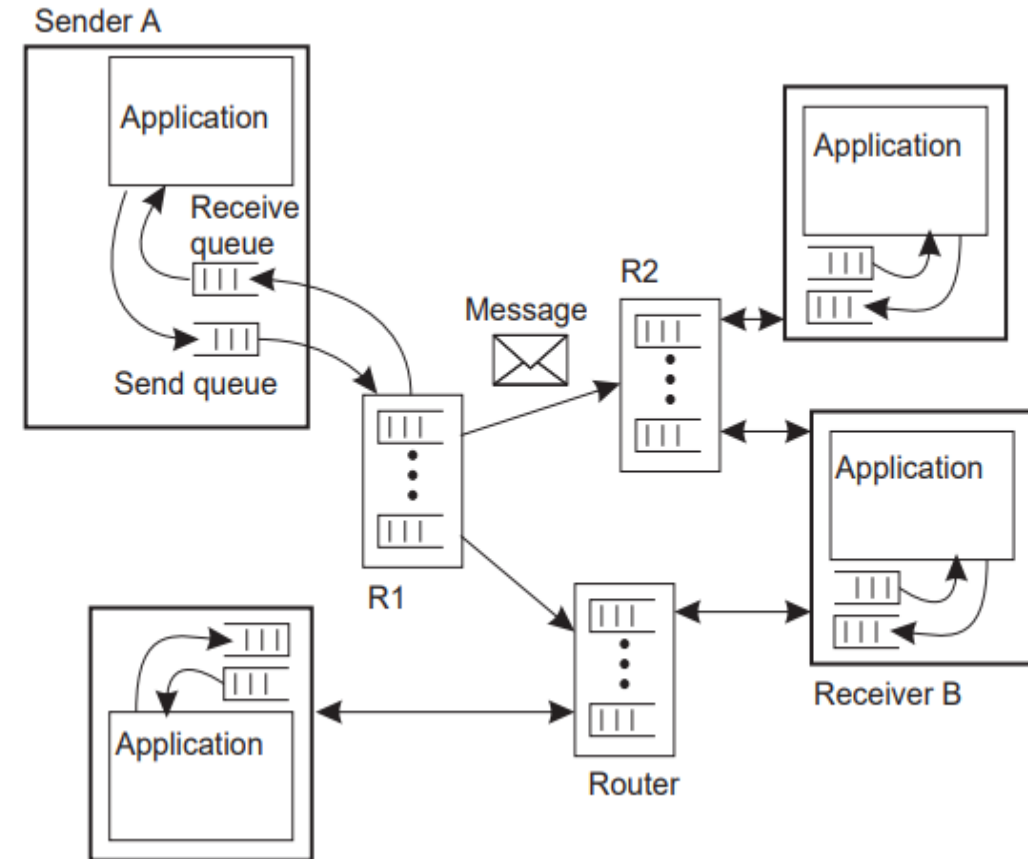
- **Provides:**

- ⇒ Persistent asynchronous communication
- ⇒ **Message Queues:** store/forward
- ⇒ Transfer of messages between queues

- In the figure, an application on sender A sends a message to an application on receiver B.
- It places the message in its local source queue, from where it is forwarded through routers R1 and R2 into the receiver's destination queue.

- **Examples:**

- ⇒ IBM MQSeries, Java Message Service, Amazon SQS, Advanced Message Queuing Protocol, MQTT, STOMP



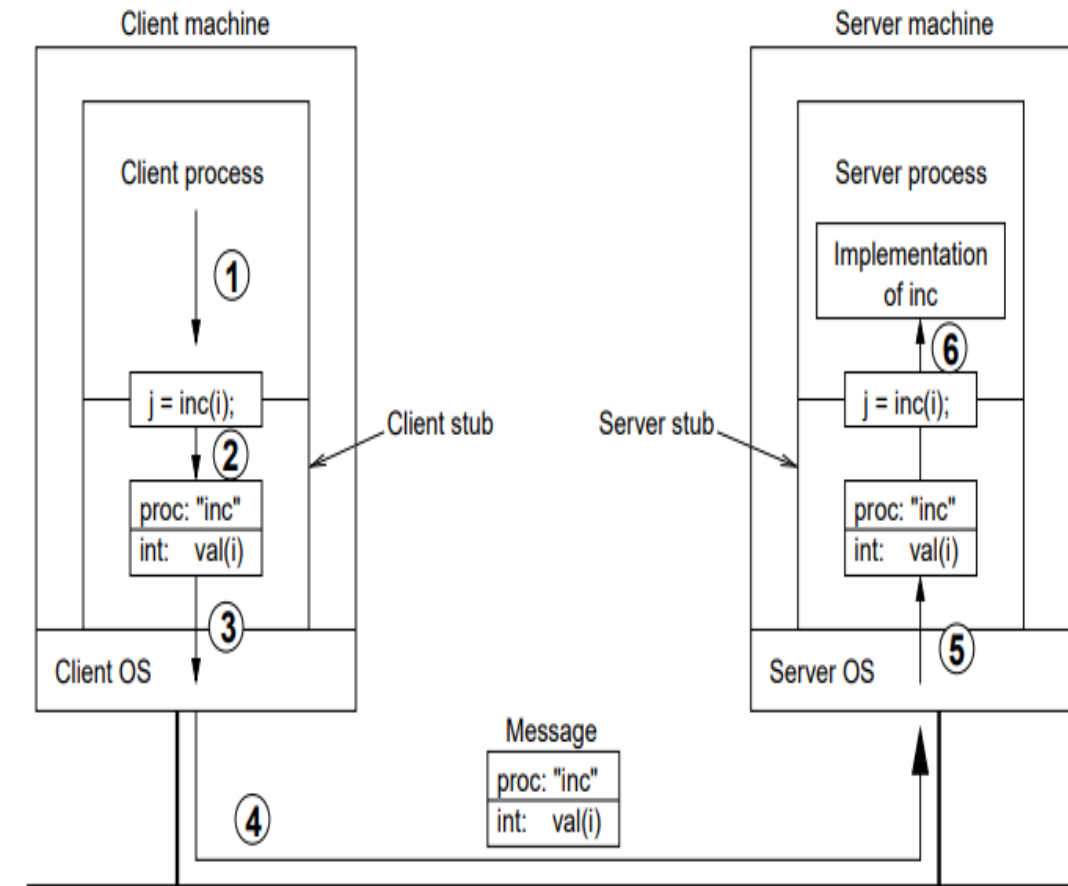
Example: Remote Procedure Call (RPC) and Remote Method Invocation (RMI)

- **RPC Concept:** Replace I/O oriented message passing model by execution of a procedure call on a remote node [BN84]:
 - ⇒ **Synchronous:** based on blocking messages
 - ⇒ Message-passing details hidden from application
 - ⇒ Procedure call parameters used to transmit data
 - ⇒ Client calls local “stub” which does messaging and marshalling
- **RMI Concept:** It is complicated for a server to keep track of the different state belonging to different clients and their invocations.
- ✓ These problems with RPC solve the remote method invocation (RMI) abstraction.
- ✓ The transition from RPC to RMI is, at its core, a transition from the server metaphor to the object metaphor.



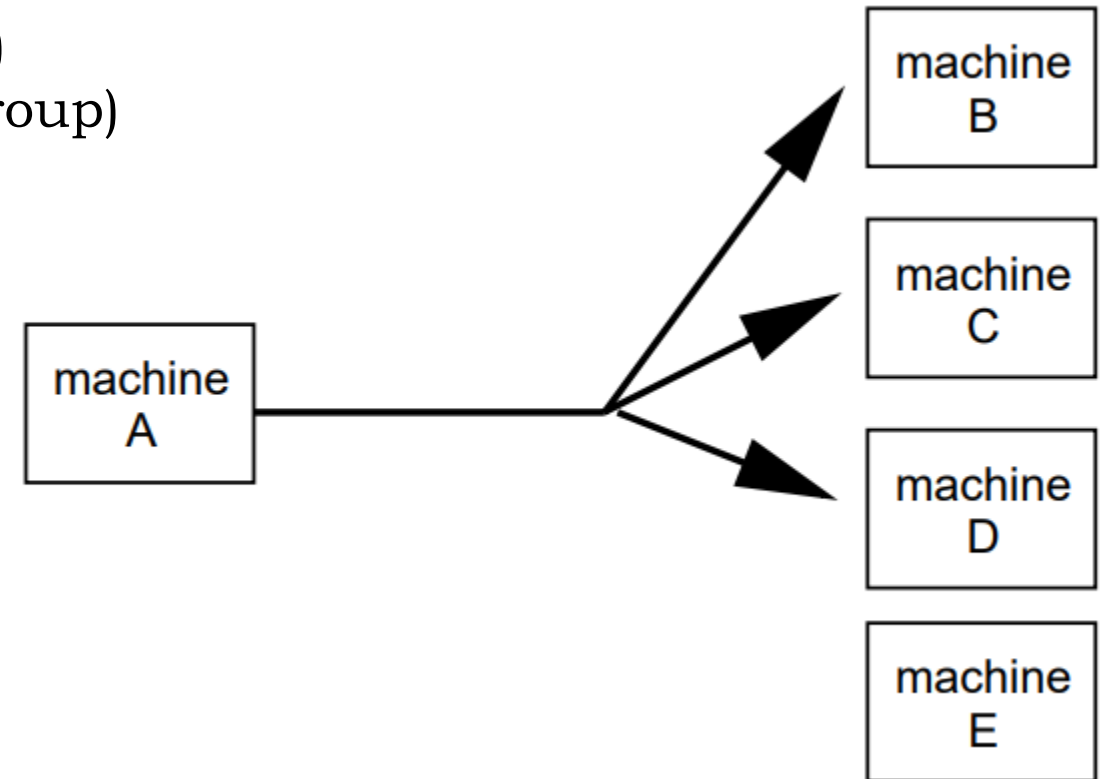
RPC Implementation

- ❶ Client calls client stub (normal procedure call)
- ❷ Client stub packs parameters into message data structure
- ❸ Client stub performs send() syscall and blocks
- ❹ Kernel transfers message to remote kernel
- ❺ Remote kernel delivers to server stub, blocked in receive()
- ❻ Server stub unpacks message, calls server (normal proc call)
- ❼ Server returns to stub, which packs result into message
- ❽ Server stub performs send() syscall
- ❾ Kernel delivers to client stub, which unpacks and returns



2. Group-based Communication

- **Two kinds of group communication:**
 - ⇒ Broadcast (message sent to everyone)
 - ⇒ Multicast (message sent to specific group)
- **Used for:**
 - ⇒ Replication of services
 - ⇒ Replication of data
 - ⇒ Service discovery
 - ⇒ Event notification
- **Issues:**
 - ⇒ Reliability
 - ⇒ Ordering
- **Example:**
 - ⇒ IP multicast
 - ⇒ Flooding



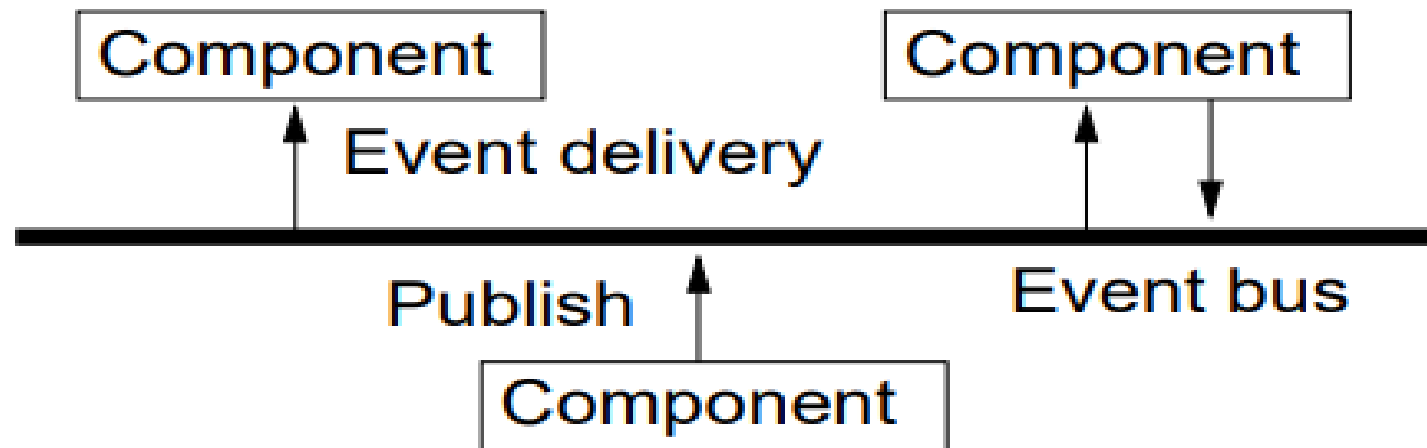
Example: Gossip-based Communication

- Technique that relies on epidemic behaviour, e.g. spreading diseases among people.
- **Variant:** rumour spreading, or gossiping.
 - ⇒ When node P receives data item x, it tries to push it to arbitrary node Q.
 - ⇒ If x is new to Q, then P keeps on spreading x to other nodes.
 - ⇒ If node Q already has x, P stops spreading x with certain probability.
- **Analogy from real life:** Spreading rumors' among people.



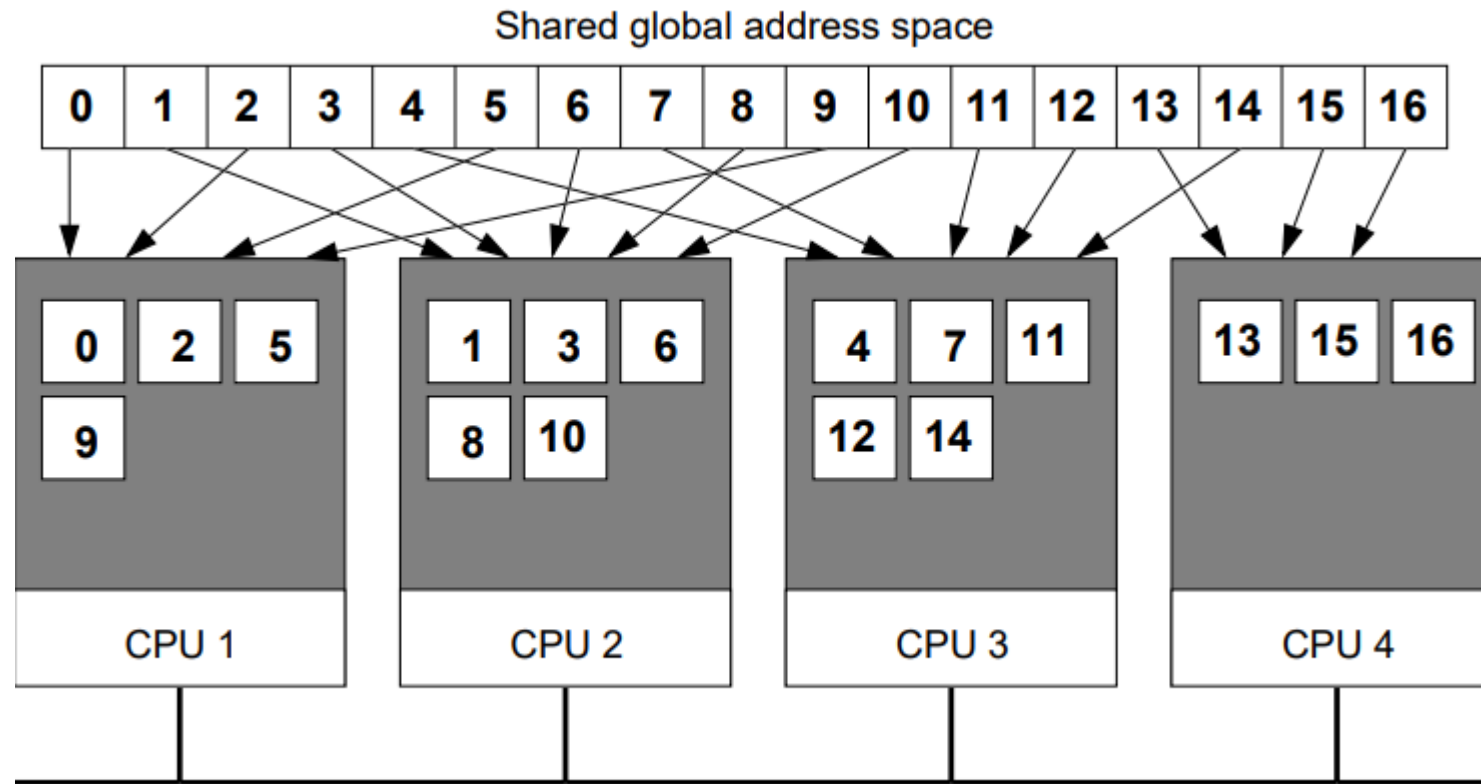
3. Event-based Communication

- Communication through propagation of events
- Generally associated with publish/subscribe systems
- Sender process publishes events
- Receiver process subscribes to events and receives only the ones it is interested in.
- **Example:** OMG Data Distribution Service (DDS), JMS, Tibc



4. Shared Space Communication

- **Example: Distributed Shared Memory:**





Thank you

